

Comparative Performance Analysis of MILP Solvers for Cryptanalysis

Halil İbrahim Kaplan¹[0009–0006–0120–6512]

Cryptanalysis Laboratory, TÜBİTAK BİLGEM, Kocaeli, Turkey
halil.kaplan@tubitak.gov.tr

Abstract. This paper provides a performance comparison of five MILP solvers applied to related-key differential cryptanalysis of ITUbee [10]. We evaluate three open-source solvers (GLPK, HiGHS, SCIP) and two commercial solvers (Gurobi, CPLEX) using MILP models for 8, 10, and 12-round attacks. As rounds increase, the number of equations and constraints grows exponentially. Experiments used an 11th Gen Intel Core i7-1165G7 processor with 32 GB of RAM. Commercial solvers (Gurobi and CPLEX) perform better than open-source options, achieving up to 94× speedup compared to GLPK for the 12-round model. This work provides guidance for choosing a solver for MILP-based cryptanalysis.

Keywords: Cryptanalysis · MILP · Differential Cryptanalysis · Block Ciphers

1 Introduction

MILP has become a standard tool for automated cryptanalysis of block ciphers, especially when searching for optimal differential and related-key characteristics. In this setting, the choice of MILP solver often determines whether an analysis is practical or infeasible, since solver runtime can vary by orders of magnitude on the same model.

This paper focuses on the performance comparison of five MILP solvers on related-key differential models for the lightweight cipher ITUbee. We benchmark three open-source solvers (GLPK, HiGHS, SCIP) and two commercial solvers (Gurobi, CPLEX) on 8-, 10-, and 12-round instances. The comparison highlights how algorithmic design, preprocessing, and search heuristics affect solver efficiency on cryptanalytic MILP workloads.

ITUbee is a software-oriented lightweight Feistel cipher operating on 80-bit blocks and an 80-bit key [10]. Its round structure includes key whitening, substitution, and linear diffusion layers, making it a suitable target for MILP-based related-key differential analysis. This paper uses ITUbee as a representative case study for solver benchmarking rather than as a proposal for a new cipher design.

The main contributions of this work are:

- a direct empirical comparison of GLPK, HiGHS, SCIP, Gurobi, and CPLEX on a series of ITUbee-related MILP models;

- an analysis of how solver performance scales as the model difficulty increases from 8 to 12 rounds;
- practical recommendations for choosing a solver for MILP-based related-key differential cryptanalysis.

The results are intended to guide cryptanalysts who need to select the most effective solver for MILP-based analysis, with a clear emphasis on performance rather than cryptographic design.

Organization of the paper. Section 2 introduces MILP-based cryptanalysis and related-key differential attacks. Section 3 describes the ITUbee cipher and the MILP model. Section 4 presents the benchmark results and analysis. Section 5 concludes with practical solver recommendations.

2 Our MILP Model

In this study, we adopt the related-key differential cryptanalysis model introduced in [9]. To facilitate a comprehensive comparison, 8-, 10-, and 12-round instances of the model are employed to evaluate the performance of various solvers. For the sake of completeness, a brief overview of related-key differential cryptanalysis and the ITUbee cipher is also provided; for a more detailed exposition, the reader is referred to [9].

2.1 ITUbee

ITUbee [10] is a lightweight software-oriented block cipher specifically designed for resource-constrained devices which include a microcontroller and have a limited battery power such as sensor nodes in wireless sensor networks.

The cipher has Feistel structure which consists of 20 rounds. It has 80 bits block size and 80 bits key size. The round structure of the cipher is given in Figure 1. Cipher also has a key whitening layer in the beginning and end of the 20 rounds. The encryption process of the cipher is given in Algorithm 1.

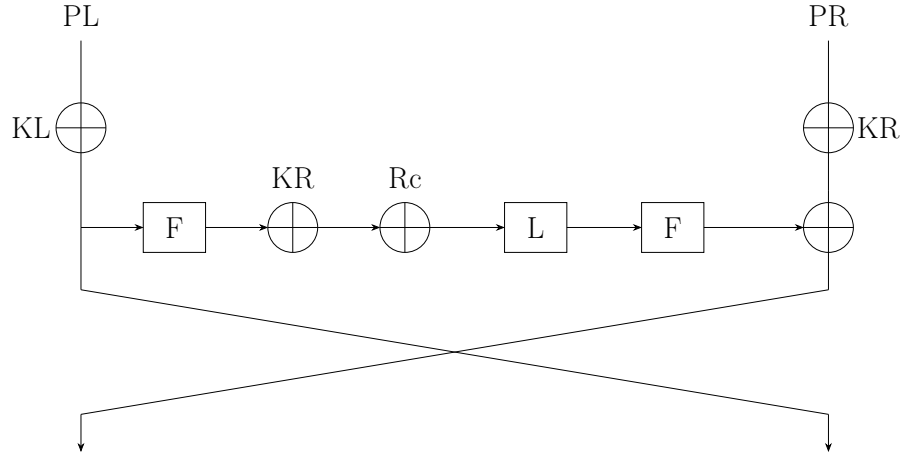
L Function The L function which has branch number 4 is computed as,
 $L(a \parallel b \parallel c \parallel d \parallel e) = (e \oplus a \oplus b) \parallel (a \oplus b \oplus c) \parallel (b \oplus c \oplus d) \parallel (c \oplus d \oplus e) \parallel (d \oplus e \oplus a)$

F Function The F function is computed as,

$$F(X) = S(L(S(X)))$$

S is the S-box used in AES [17]. S is the only non-linear structure in the ITUbee algorithm. 40 bit input of S is splitted into 8 bits then enters the s-box.

$$S(a \parallel b \parallel c \parallel d \parallel e) = s[a] \parallel s[b] \parallel s[c] \parallel s[d] \parallel s[e]$$

**Fig. 1.** ITUbee round structure**Algorithm 1** Pseudo code for ITUbee encryption

```

1:  $X_1 \leftarrow P_L \oplus K_L$ 
2:  $X_0 \leftarrow P_R \oplus K_R$ 
3: for  $i = 1$  to 20 do
4:   if  $i \in \{1, 3, \dots, 19\}$  then
5:      $RK \leftarrow K_R$ 
6:   else
7:      $RK \leftarrow K_L$ 
8:   end if
9:    $X_{i+1} \leftarrow X_{i-1} \oplus F(L(RK \oplus RC_i \oplus F(X_i)))$ 
10:  Round constant is added to the rightmost 16 bits
11: end for
12:  $C_L \leftarrow X_{20} \oplus K_R$ 
13:  $C_R \leftarrow X_{21} \oplus K_L$ 

```

Key Schedule ITUbee has no key schedule. The algorithm splits the 80 bit key into two 40 bit parts K_L and K_R and uses the right part of the key in odd rounds and the left part of the keys in even rounds.

2.2 Related-key Differential Cryptanalysis

Related-key differential cryptanalysis is an extension of classical differential cryptanalysis in which the adversary exploits known relationships between different secret keys. While differential cryptanalysis was originally introduced by Eli Biham and Adi Shamir [2], the related-key attack model was later formalized by Kelsey, Schneier, and Wagner [12]. In this model, the attacker is allowed to observe encryptions under multiple keys that are related through a known difference, and analyzes how both data and key differences propagate through the

cipher. The main idea is to construct high-probability differential characteristics that incorporate key differences, enabling the extraction of information about the secret key. This approach is particularly effective against block ciphers with weak or simple key schedule algorithms, where key differences propagate in a predictable manner.

For the related-key model, we must explicitly represent the key differences in addition to the data differences. To do this, we introduce binary variables for key activity and add constraints that force at least one key byte to be active when a related-key difference is present. Without these constraints, the model could avoid key activity and would effectively reduce to the ordinary differential model.

This related-key model also contains more intermediate variables than the differential and linear models, because we must represent the extra variables that appear after the key XOR operation in the middle of the round. The modeling of XOR, linear, and key-dependent components follows the MILP approach of Mouha and Preneel [14]. A sketch of the one-round related-key MILP model is shown in Figure 2.

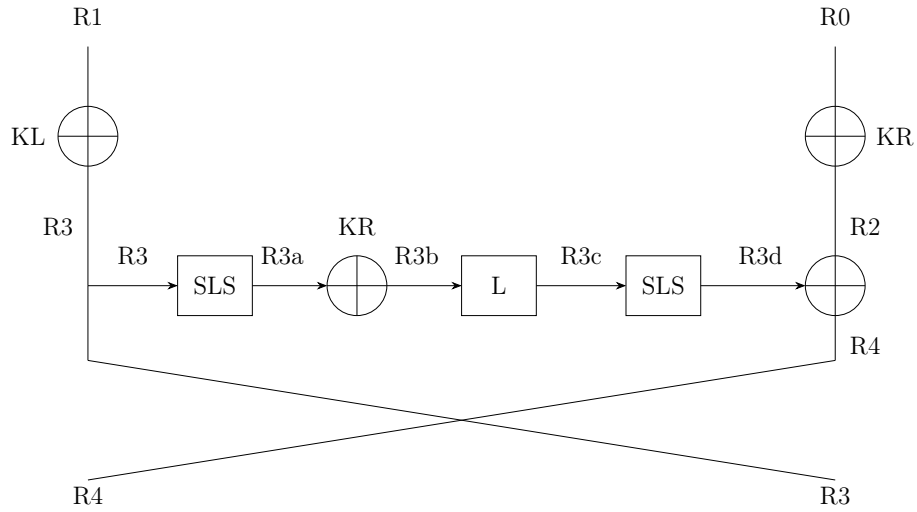


Fig. 2. ITUbee MILP Related-Key Differential Cryptanalysis Sketch

We found that 8 round of the algorithm has minimum 16 active s-box. ITUbee algorithm uses AES's [4] s-box. AES s-box have best input-output difference 2^{-6} . Therefore best probability for differential characteristic will be $(2^{-6})^{16} = 2^{-96}$ which is not usable for related key differential attack. Authors of the ITUbee algorithm claims in [10] that algorithm is secure against related key differential cryptanalysis after 10 round. In our model we found that 8 round is enough for

security against related key differential cryptanalysis. Best 8 round characteristic found with the model can be seen in Table 1

Stage	Binary Index	Characteristic
Key	KL	{0: 0.0, 1: 1.0, 2: 0.0, 3: 1.0, 4: 0.0}
	KR	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
1. round input	R1	{0: 0.0, 1: 1.0, 2: 0.0, 3: 1.0, 4: 0.0}
	R0	{0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
1. round middle values	R2	{0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
	R3-3a-3b-3c-3d	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
2. round input	R4	{0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
	R3	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
2. round middle values	R4a	{0: 0.0, 1: 1.0, 2: 0.0, 3: 1.0, 4: 0.0}
	R4b-4c-4d	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
3. round input	R5	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
	R4	{0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
3. round middle values	R5a-5b-5c-5d	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
4. round input	R6	{0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
	R5	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
4. round middle values	R6a	{0: 0.0, 1: 1.0, 2: 0.0, 3: 1.0, 4: 0.0}
	R6b-6c-6d	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
5. round input	R7	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
	R6	{0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
5. round middle values	R7a	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
	R7b-7c-7d	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
6. round input	R8	{0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
	R7	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
6. round middle values	R8a	{0: 0.0, 1: 1.0, 2: 0.0, 3: 1.0, 4: 0.0}
	R8b-8c-8d	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
7. round input	R9	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
	R8	{0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
7. round middle values	R9a-9b-9c-9d	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
8. round input	R10	{0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}
	R9	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
8. round middle values	R10a	{0: 0.0, 1: 1.0, 2: 0.0, 3: 1.0, 4: 0.0}
	R10b-10c-10d	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
9. round input	R11	{0: 0.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
	R10	{0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0, 4: 1.0}

Table 1. Best 8 round related-key differential characteristic of ITUbee algorithm

3 Solver Specifications

This section provides a detailed description of the mixed-integer linear programming (MILP) solvers evaluated in our comparative study, with emphasis on their

underlying optimization techniques. Modern MILP solvers rely on a combination of linear programming (LP) methods, such as the simplex algorithm [3] and interior-point methods [11], together with tree-based search strategies, most notably branch-and-bound [13] and its extensions. These approaches are typically enhanced with cutting-plane techniques [6], resulting in the widely used branch-and-cut framework [15, 18].

3.1 GLPK (GNU Linear Programming Kit)

GLPK is an open-source linear and integer programming solver developed within the GNU project [16]. In our experiments, version 4.65 is used. For LP problems, GLPK employs the revised simplex method [3], while MILP instances are handled via a branch-and-cut framework that combines branch-and-bound [13] with cutting planes [6]. Although GLPK provides deterministic and reliable solutions, it lacks many advanced heuristics and parallelization features found in modern commercial solvers. Consequently, its performance is generally limited on large-scale MILP instances, but it remains a widely used baseline in academic research due to its accessibility and transparency.

3.2 HiGHS (High-performance Interior-point Solver)

HiGHS is a modern open-source optimization solver developed at the University of Edinburgh [5]. Version 0.9 is used in this study. It incorporates multiple LP solution techniques, including dual and primal simplex methods [3] as well as interior-point methods [11]. For MILP problems, HiGHS implements a parallel branch-and-bound framework [13], enhanced with presolve reductions and heuristic search strategies. Its design emphasizes computational efficiency and scalability, making it competitive with more established solvers, particularly in multi-core environments.

3.3 SCIP (Solving Constraint Integer Programs)

SCIP is an advanced open-source framework for constraint integer programming developed by the Zuse Institute Berlin [1]. Version 7.0 is used in our experiments. SCIP extends the classical MILP paradigm by integrating constraint programming techniques with branch-and-cut methods [15]. It employs a branch-and-bound search tree [13], strengthened by cutting planes [6], primal heuristics, and symmetry-breaking mechanisms. Additionally, SCIP supports nonlinear and quadratic constraints, making it suitable for a broader class of optimization problems. Among open-source solvers, SCIP is often regarded as one of the most powerful due to its flexibility and advanced algorithmic components.

3.4 Gurobi Optimizer

Gurobi is a state-of-the-art commercial optimization solver developed by Gurobi Optimization [7]. Version 10.0 is used in this work. It utilizes a highly optimized

branch-and-cut framework [15, 18], combined with advanced presolve techniques, cutting-plane separation, and primal-dual heuristics. For LP relaxations, Gurobi supports both simplex methods [3] and barrier (interior-point) methods [11]. The solver is designed to exploit parallel architectures efficiently and provides extensive parameter tuning capabilities. Due to its performance and robustness, Gurobi is widely used as a benchmark in both academic and industrial applications.

3.5 CPLEX (IBM CPLEX Optimization Studio)

CPLEX is a commercial optimization solver developed by IBM with a long history of algorithmic development [8]. Version 12.10 is considered in this study. It supports multiple LP solution strategies, including primal and dual simplex methods [3] and barrier methods with crossover [11]. For MILP problems, CPLEX employs a branch-and-cut approach [15], integrating cutting planes [6], preprocessing, and probing techniques. It also incorporates advanced heuristics and parallel processing capabilities. CPLEX remains one of the most widely used solvers in enterprise-level optimization and large-scale industrial applications.

3.6 Experimental Configuration

All solvers were executed with default parameters to ensure fair and reproducible comparison. This approach reflects typical usage patterns where practitioners rely on default settings without extensive parameter tuning. Each solver was allowed unlimited computation time (no time limits were imposed) to find optimal solutions. Experiments were conducted on an 11th Gen Intel Core i7-1165G7 processor operating at 2.80 GHz with 32 GB of system RAM, running Ubuntu Linux. This ensures all solvers had equal access to computational resources, with no artificial constraints or advantages.

4 Results and Analysis

Comprehensive benchmarking was conducted on MILP models for 8, 10, and 12-round related-key differential attacks on ITUbee. The results demonstrate clear performance distinctions between solver categories, with significant implications for practical related-key differential cryptanalysis.

4.1 8-Round Results

Our 8-round model consists of 687 variables and 2147 constraints. Our results show that GLPK, the baseline open-source solver, required 257 seconds to find a solution. HiGHS improved on this with a time of 231 seconds, while SCIP further reduced the time to 86 seconds. The commercial solvers Gurobi and CPLEX achieved faster times of 37 and 16 seconds, respectively, representing speedups of $6.9\times$ and $16.0\times$ compared to GLPK (see Table 2).

Table 2. Solution times for the 8-round model

Solver	Time (s)	Speedup
GLPK	257	1.0×
HiGHS	231	1.1×
SCIP	86	2.9×
Gurobi	37	6.9×
CPLEX	16	16.0×

4.2 10-Round Results

The 10-round model consists of 945 variables and 3182 constraints. In this harder instance, GLPK required 1610 seconds to solve the problem. HiGHS improved substantially, finishing in 321 seconds, and SCIP further reduced the time to 202 seconds. The commercial solvers again delivered the best performance, with Gurobi solving the model in 34 seconds and CPLEX in 40 seconds, representing speedups of 47.4× and 40.3× versus GLPK, respectively (see Table 3).

Table 3. Solution times for the 10-round model

Solver	Time (s)	Speedup
GLPK	1610	1.0×
HiGHS	321	5.0×
SCIP	202	7.9×
Gurobi	34	47.4×
CPLEX	40	40.3×

4.3 12-Round Results

The 12-round model is the most challenging benchmark, with 1203 variables and 4217 constraints. GLPK required 4543 seconds to solve this instance, while HiGHS completed in 1780 seconds. SCIP again offered a strong open-source alternative at 376 seconds. Gurobi and CPLEX achieved the fastest times of 48 and 53 seconds, corresponding to speedups of 94.6× and 85.7× relative to GLPK (Table 4).

Table 4. Solution times for the 12-round model

Solver	Time (s)	Speedup
GLPK	4543	1.0×
HiGHS	1780	2.5×
SCIP	376	12.0×
Gurobi	48	94.6×
CPLEX	53	85.7×

4.4 Discussion

The experimental results reveal several important trends:

1. **Commercial vs. Open-Source Gap:** Commercial solvers (Gurobi and CPLEX) consistently outperform open-source alternatives, with speedups increasing dramatically for larger problem sizes. For the 12-round model, Gurobi achieves a $94.6\times$ speedup over GLPK.
2. **Problem Scaling:** As the problem complexity increases (8 to 12 rounds), the performance gap between solvers widens. This suggests that the commercial solvers use methods that are more effective on harder instances.
3. **Open-Source Alternatives:** Among open-source solvers, SCIP consistently outperforms both GLPK and HiGHS, achieving $2.9\times$ speedup on 8-rounds and $12\times$ speedup on 12-rounds compared to GLPK.
4. **Commercial Solver Parity:** Gurobi and CPLEX show comparable performance, with Gurobi slightly faster on 10 and 12-round models, while CPLEX excels on the 8-round problem.

The timing analysis demonstrates that solver choice is critical for practical cryptanalysis. For resource-constrained deployments, SCIP provides a reasonable alternative to commercial solvers. However, for optimal performance, commercial solutions are strongly recommended.

5 Conclusion

This paper provides a comparison of five MILP solvers in the context of cryptanalytic applications, specifically related-key differential attacks on the ITUbee lightweight block cipher. Our findings show that solver selection has a large effect on the feasibility of MILP-based related-key differential cryptanalysis.

Key conclusions include:

1. Commercial MILP solvers (Gurobi and CPLEX) are more efficient than open-source alternatives and can be preferred for larger, time-sensitive analyses.
2. For organizations without access to commercial solvers, SCIP is a good open-source option that performs well on moderate problem sizes.
3. The performance advantage of commercial solvers grows with problem complexity, suggesting that their methods are more effective on harder instances.
4. Cryptanalysts should evaluate solver options based on their constraints (computational budget, licensing availability, problem size) when conducting related-key differential cryptanalysis.

References

1. Berlin, Z.I.: Scip: Solving constraint integer programs. <https://www.scipopt.org/> (2023)

2. Biham, E., Shamir, A.: Differential cryptanalysis of des-like cryptosystems. In: Workshop on the Theory and Application of Cryptographic Techniques. pp. 2–21. Springer (1990)
3. Dantzig, G.B.: Linear programming and extensions. Princeton university press (1963)
4. Dworkin, M.J.: The advanced encryption standard (aes) and its use in cryptographic standards. NIST Special Publication **800**, 38 (2001)
5. of Edinburgh, U.: Highs: High-performance interior-point solver. <https://github.com/ERGO-Code/HiGHS> (2023)
6. Gomory, R.E.: Outline of an algorithm for integer solutions to linear programs. Bulletin of the American Mathematical Society **64**(5), 275–278 (1958)
7. Gurobi Optimization, L.: Gurobi optimizer. <https://www.gurobi.com/> (2024)
8. IBM: Ibm cplex optimization studio. <https://www.ibm.com/products/ilog-cplex-optimization-studio> (2024)
9. İbrahim Kaplan, H.: Usage of mixed integer linear programming in cryptanalysis of block ciphers. Cryptology ePrint Archive, Paper 2026/224 (2026), <https://eprint.iacr.org/2026/224>
10. Karakoç, F., Demirci, H., Harmancı, A.E.: Itubee: a software oriented lightweight block cipher. In: Lightweight Cryptography for Security and Privacy: Second International Workshop, LightSec 2013, Gebze, Turkey, May 6-7, 2013, Revised Selected Papers 2. pp. 16–27. Springer (2013)
11. Karmarkar, N.: A new polynomial-time algorithm for linear programming. Combinatorica **4**(4), 373–395 (1984)
12. Kelsey, J., Schneier, B., Wagner, D.: Related-key cryptanalysis of 3-way, bihamdes, cast, des-x, newdes, rc2, and tea. In: International Conference on Information and Communications Security. pp. 233–246. Springer (1996)
13. Land, A.H., Doig, A.G.: An automatic method of solving discrete programming problems. In: Econometrica: journal of the Econometric Society. pp. 497–520. JSTOR (1960)
14. Mouha, N., Preneel, B.: Differential and linear cryptanalysis using mixed-integer linear programming. In: Information Security and Cryptology – ICISC 2012. Lecture Notes in Computer Science, vol. 7839, pp. 52–67. Springer (2012). https://doi.org/10.1007/978-3-642-36392-4_5
15. Nemhauser, G.L., Wolsey, L.A.: Integer and combinatorial optimization. John Wiley & Sons (1988)
16. Project, G.: Glpk (gnu linear programming kit). <https://www.gnu.org/software/glpk/> (2024)
17. of Standards, N.I., Technology: Advanced encryption standard (aes) (2001), <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf>
18. Wolsey, L.A.: Integer programming. John Wiley & Sons (1998)